
系统开发工具基础

实验报告

实验周次：第 2 周

学 院 信息科学与工程学部

专 业 计算机科学与技术

姓 名 臧文商

学 号 25020007160

实验日期 2026.8.31

一、 课堂练习

1. 题目一

(1) 题目内容

第 5 题 控制一个可清理的后台任务

主题：命令行环境：进程、信号与任务控制 建议用时：12—15 分钟

将下方脚本保存为 q05/worker.sh，再完成任务控制。

给定内容

```
#!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
    echo "$n"
    n=$((n+1))
    sleep 1
done
```

题目要求

- 赋予脚本执行权限，在前台启动，并把 stdout、stderr 分别写入 stdout.log、stderr.log。
- 使用 Ctrl-Z 挂起任务，再用 bg 让它在后台继续；使用 jobs 确认状态。
- 使用 jobs -p 或等价方式取得 PID，不得手工抄写 PID；发送 SIGTERM 并等待任务结束。
- 确认 cleanup.log 中出现 CLEAN_EXIT，且任务已经退出；禁止使用 kill -9。

(2) 解题过程

克隆脚本，使用 chmod +x 赋予脚本执行权限，运行脚本将 stdout、stderr 分别写入 stdout.log、stderr.log，使用 Ctrl+Z 挂起任务，用 bg 让它在后台继续，用 jobs 查看任务状态，jobs -p 获取 PID，kill -s SIGTERM 发送终止信号，cat 查看 cleanup.log 验证清理逻辑。

```
guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q05 (main)
$ vi worker.sh

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q05 (main)
$ chmod +x worker.sh

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q05 (main)
$ ./worker.sh > stdout.log 2> stderr.log

[1]+  Stopped                  ./worker.sh > stdout.log 2> stderr.log

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q05 (main)
$ bg
[1]+  ./worker.sh > stdout.log 2> stderr.log &

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q05 (main)
$ jobs
[1]+  Running                  ./worker.sh > stdout.log 2> stderr.log &

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q05 (main)
$ jobs -p
1670

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q05 (main)
$ SIGTERM
bash: SIGTERM: command not found

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q05 (main)
$ kill -s SIGTERM $(jobs -p)

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q05 (main)
$ cat cleanup.log
CLEAN_EXIT
[1]+  Done                    ./worker.sh > stdout.log 2> stderr.log
```

(3) 实验结果

完成任务控制，如上图。

2. 题目二

(1) 题目内容

第 6 题 语义重构与本地开发反馈

主题：开发环境与工具 建议用时：12—15 分钟

在 q06 中按照给定代码创建三个 Python 文件，并使用编辑器语言服务器完成一次语义重构。

给定内容

```
# math_utils.py
def total_price(price: float, count: int) -> float:
    return price * count

# app.py
from math_utils import total_price
print(total_price(12.5, 4))

# test_math_utils.py
from math_utils import total_price
def test_total_price():
    assert total_price(12.5, 4) == 50.0
```

题目要求

第 4 页

系统开发工具基础 · 每周课上实验检查题

- 在已配置好 Python、pytest 和 ruff 的课程环境中打开 q06，并启用 Python 语言服务器。
- 分别演示跳转到定义和查找引用。
- 使用“重命名符号”把 total_price 改为 calculate_total，保证定义和所有代码引用同步改变。
- 在 app.py 中临时加入未使用的 import os，用 ruff 发现并删除；最后运行 ruff 和 pytest 并保证通过。

(2) 解题过程

先配置 pytest 和 ruff

```
guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q06 (main)
$ python -m pip install pytest ruff
Collecting pytest
  Downloading pytest-8.4.2-py3-none-any.whl (365 kB)
    _____ 365.8/365.8 KB 437.8 kB/s eta 0:00:00
Collecting ruff
  Downloading ruff-0.16.5-py3-none-win_amd64.whl (10.5 MB)
    _____ 10.5/10.5 MB 3.4 MB/s eta 0:00:00
Collecting tomli>=1
  Downloading tomli-2.4.1-py3-none-any.whl (14 kB)
Collecting pygments>=2.7.2
  Downloading pygments-2.21.0-py3-none-any.whl (1.3 MB)
    _____ 1.3/1.3 MB 6.6 MB/s eta 0:00:00
```

验证环境并建立初始文件

```
guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q06 (main)
$ python -m pytest --version
pytest 8.4.2

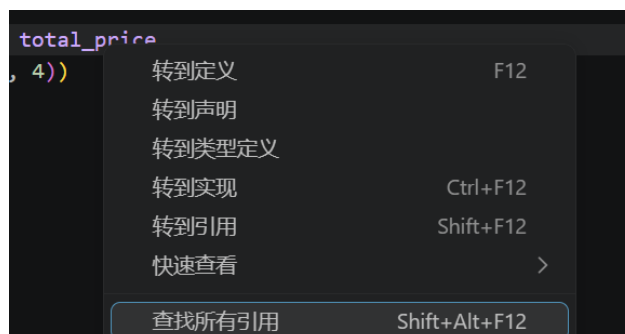
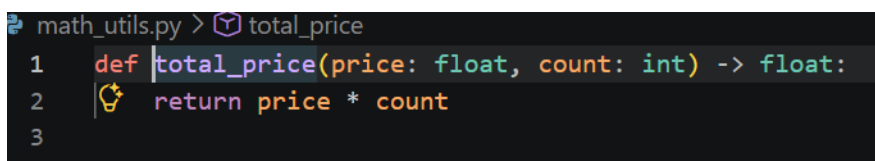
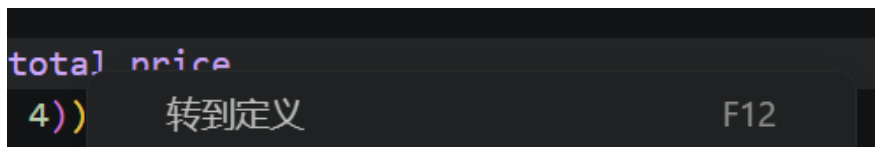
guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q06 (main)
$ python -m ruff --version
ruff 0.16.5

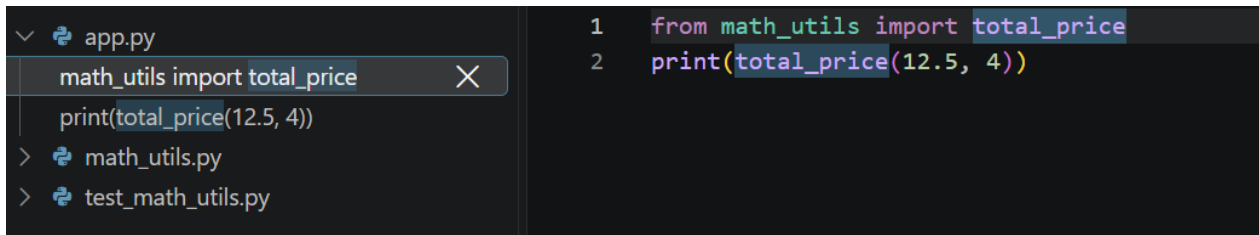
guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q06 (main)
$ vi math_utils.py

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q06 (main)
$ vi app.py

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q06 (main)
$ vi test_math_utils.py
```

使用 VS Code 打开项目，启用 Python 语言服务器，分别进行“跳转到定义”“查找引用”和“重命名”操作如下图





```

1 from math_utils import total_price
2 print(total_price(12.5, 4))

```



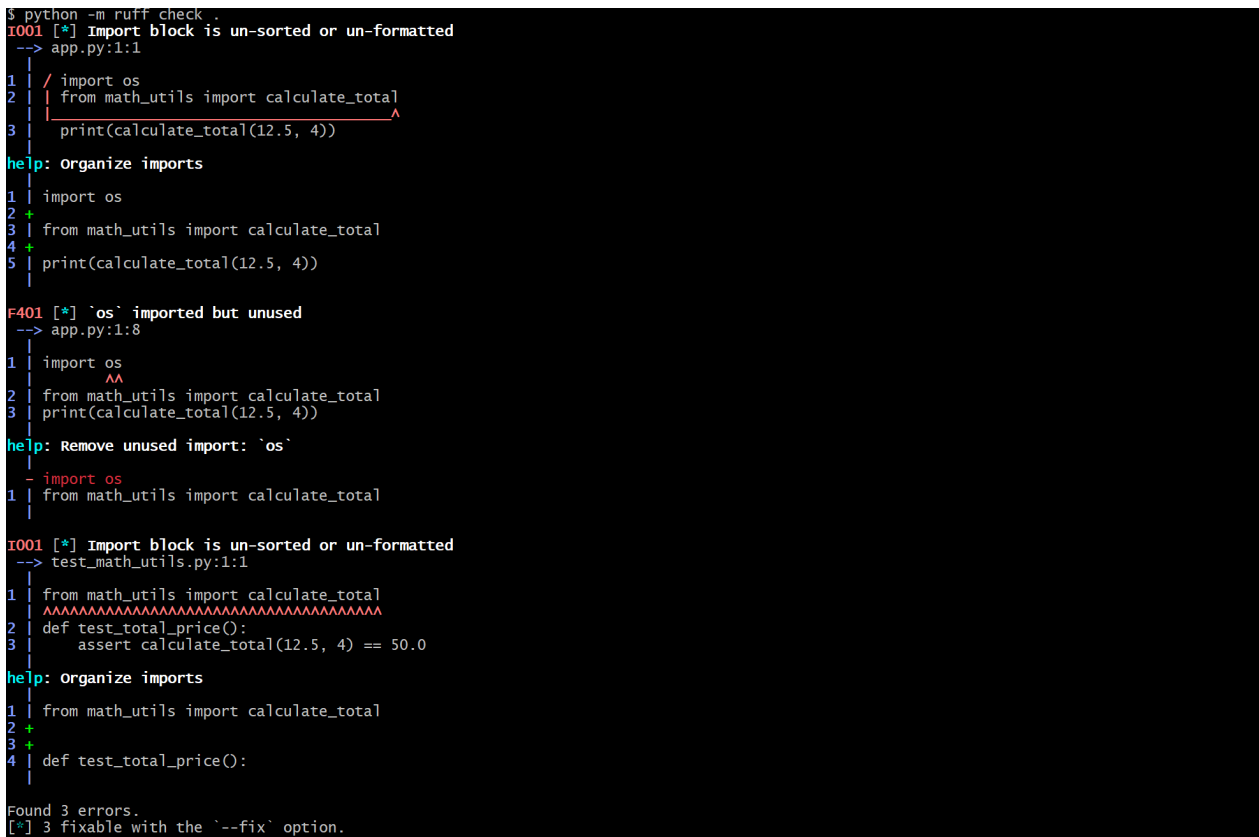
```

1 def total_price(price: float, count: int) -> float:
2
3

```

calculate_total
按 Enter 进行重命名, 按 Ctrl+Enter 进行预览

在 app.py 中加入未使用的 import.os, 用 ruff 发现



```

$ python -m ruff check .
I001 [*] Import block is un-sorted or un-formatted
--> app.py:1:1
1 | / import os
2 | | from math_utils import calculate_total
3 | | print(calculate_total(12.5, 4))
help: Organize imports
1 | import os
2 | +
3 | from math_utils import calculate_total
4 | +
5 | print(calculate_total(12.5, 4))

F401 [*] `os` imported but unused
--> app.py:1:8
1 | import os
2 |   ^
3 |   ^
help: Remove unused import: `os`
1 | - import os
2 |   from math_utils import calculate_total

I001 [*] Import block is un-sorted or un-formatted
--> test_math_utils.py:1:1
1 | from math_utils import calculate_total
2 | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
3 | def test_total_price():
4 |     assert calculate_total(12.5, 4) == 50.0
help: Organize imports
1 | from math_utils import calculate_total
2 | +
3 | +
4 | def test_total_price():

Found 3 errors.
[*] 3 fixable with the '--fix' option.

```

用 ruff 修正后, 运行 ruff 和 pytest。

```
guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q06 (main)
$ python -m ruff check --fix .
Found 3 errors (3 fixed, 0 remaining).
```

```
guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q06 (main)
$ python -m ruff check .
All checks passed!
```

```
guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q06 (main)
$ python -m pytest
```

```
===== test session starts =====
```

```
platform win32 -- Python 3.9.13, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\Users\guolicheng\Desktop\系统开发工具基础\实验\week2\q06
collected 1 item
```

```
test_math_utils.py .
```

```
===== 1 passed in 0.01s =====
```

(3) 实验结果

成功完成跳转定义、查找引用、重命名等操作；加入 `import os` 后用 `ruff` 发现并删除，最后运行 `ruff` 检查无报错，`pytest` 单元测试全部通过。

3. 题目三

(1) 题目内容

第 7 题 用调试器定位归并排序缺陷

主题: Debugging 建议用时: 12—15 分钟

将下列存在缺陷的归并排序代码保存为 q07/merge_sort.py, 再使用调试器定位并修复。

给定内容

```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1 # 此处存在缺陷
    return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) <= 1: return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
```

题目要求

- 先运行给定输入, 确认结果错误或出现异常。
- 使用 pdb、IDE debugger 或等价调试器, 在 merge 中设置断点并观察 i、j、left[i]、right[j]。
- 完成最小修复, 不得用 sorted 替换归并排序。
- 使用 pytest 增加两个测试: 给定输入和含重复元素的列表; 保证测试通过。

(2) 解题过程

先运行给定输入, 确认出现异常。

```
guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q07 (main)
$ python merge_sort.py
Traceback (most recent call last):
  File "C:\Users\guolicheng\Desktop\系统开发工具基础\实验\week2\q07\merge_sort.py", line 17, in <module>
    print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
  File "C:\Users\guolicheng\Desktop\系统开发工具基础\实验\week2\q07\merge_sort.py", line 15, in merge_sort
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))
  File "C:\Users\guolicheng\Desktop\系统开发工具基础\实验\week2\q07\merge_sort.py", line 15, in merge_sort
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))
  File "C:\Users\guolicheng\Desktop\系统开发工具基础\实验\week2\q07\merge_sort.py", line 4, in merge
    while i < len(left) and j < len(right):
TypeError: object of type 'NoneType' has no len()
```

使用 VS Code, 在 merge 中设置断点并观察 i,j,left[i],right[j], 发现异常来源于: else 分支提前

return ; 部分分支无返回; right 数组下标错用 i, 应使用 j。

```

1 def merge(left, right):
2     result = []
3     i = j = 0
4     while i < len(left) and j < len(right):
5         if left[i] <= right[j]:
6             result.append(left[i]); i += 1
7         else:
8             return result + left[i:] + right[j:]
9             result.append(right[i]); j += 1 # 此处存在缺陷
10
11 def merge_sort(a):
12     if len(a) <= 1:
13         return a
14     m = len(a) // 2
15     return merge(merge_sort(a[:m]), merge_sort(a[m:]))
16
17 print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
18

```

变量

- Locals
- Globals

监视

- i = 0
- j = 0
- left[i] = 3
- right[j] = 1

针对上述问题进行最小修复如下图

```

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1 # 此处存在缺陷
    return result+left[i:]+right[j:]

def merge_sort(a):
    if len(a) <= 1:
        return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))

```

使用 pytest 增加两个测试。

```

from merge_sort import merge_sort

# 题目给定输入
def test_given_input():
    assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1, 1, 2, 3, 4, 5, 6, 9]

# 含重复元素的列表
def test_dup_elements():
    assert merge_sort([5, 2, 2, 8, 2, 1]) == [1, 2, 2, 2, 5, 8]

```

(3) 实验结果

测试通过，异常解决。

```
guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q07 (main)
$ python merge_sort.py
[1, 1, 2, 3, 4, 5, 6, 9]

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q07 (main)
$ vi test_merge.py

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q07 (main)
$ python -m pytest test_merge.py -v
===== test session starts =====
platform win32 -- Python 3.9.13, pytest-8.4.2, pluggy-1.6.0 -- C:\Users\guolicheng\AppData\Local\Programs\Python\Python39\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\guolicheng\Desktop\系统开发工具基础\实验\week2\q07
collected 2 items

test_merge.py::test_given_input PASSED [ 50%]
test_merge.py::test_dup_elements PASSED [100%]

===== 2 passed in 0.02s =====
```

4. 题目四

(1) 题目内容

第 8 题 先测量，再优化慢速词频程序

主题: Profiling 建议用时: 12—15 分钟

在 q08 中创建可重复生成的词表和故意低效的去重程序，再使用性能分析工具优化。

给定内容

```
# generate_words.py
import random
```

第 5 页

系统开发工具基础 · 每周课上实验检查题

```
random.seed(20260831)
vocab = [f"w{i}" for i in range(1000)]
with open("words.txt", "w", encoding="utf-8") as f:
    f.write(" ".join(random.choice(vocab) for _ in range(30000)))

# wordfreq.py
words = open("words.txt", encoding="utf-8").read().split()
unique = []
for word in words:
    if word not in unique:
        unique.append(word)
print("count=", len(unique))
```

题目要求

- 生成 words.txt，使用 time 运行原程序 2 次并记录耗时。
- 使用 cProfile -s cumulative 运行一次，找出主要耗时位置。
- 在不改变最终 count 的前提下优化程序；不得写死 1000。
- 使用相同输入再运行 2 次，计算优化前后中位数和加速比。

(2) 解题过程

建立初始文档后，运行 generate_words.py 生成 words.txt，使用 time 运行原程序两次，记录耗时分别为 1.635s 和 1.639s。

```

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q08 (main)
$ python generate_words.py

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q08 (main)
$ time python wordfreq.py
count= 1000

real    0m1.635s
user    0m0.000s
sys     0m0.062s

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q08 (main)
$ time python wordfreq.py
count= 1000

real    0m1.629s
user    0m0.000s
sys     0m0.046s

```

使用 cProfile-s cumulative 运行一次，找出主要耗时位置为列表 append 操作与列表成员判断。

```

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q08 (main)
$ python -m cProfile -s cumulative wordfreq.py
count= 1000
1012 function calls in 0.101 seconds

ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
1      0.000    0.000    0.101    0.101 {built-in method builtins.exec}
1      0.099    0.099    0.101    0.101 wordfreq.py:1(<module>)
1      0.001    0.001    0.001    0.001 {method 'split' of 'str' objects}
1      0.000    0.000    0.000    0.000 {method 'read' of '_io.TextIOWrapper' objects}
1      0.000    0.000    0.000    0.000 {built-in method io.open}
1      0.000    0.000    0.000    0.000 {built-in method builtins.print}
1      0.000    0.000    0.000    0.000 codecs.py:319(decode)
1000    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
1      0.000    0.000    0.000    0.000 {built-in method _codecs.utf_8_decode}
1      0.000    0.000    0.000    0.000 codecs.py:309(__init__)
1      0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
1      0.000    0.000    0.000    0.000 codecs.py:260(__init__)
1      0.000    0.000    0.000    0.000 {built-in method builtins.len}

```

将存储容器由列表改为集合，利用集合哈希查找降低成员判断时间复杂度，完成程序优化。

```

words = open("words.txt", encoding="utf-8").read().split()
unique = set()
for word in words:
    unique.add(word)
print("count=", len(unique))

```

使用相同输入再运行两次，记录运行时间分别为 0.535s 和 0.520s

```
guoli cheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q08 (main)
$ vi wordfreq.py

guoli cheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q08 (main)
$ time python wordfreq.py
count= 1000

real    0m0.535s
user    0m0.015s
sys     0m0.031s

guoli cheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q08 (main)
$ MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q08 (main)
$ time python wordfreq.py
bash: syntax error near unexpected token `('
bash: $: command not found

guoli cheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q08 (main)
$ time python wordfreq.py
count= 1000

real    0m0.520s
user    0m0.015s
sys     0m0.015s
```

(3) 实验结果

计算得优化前运行时间中位数为 1.637s，优化后运行时间中位数为 0.5275s，加速比为 3.1。

二、 课后练习

1. 题目一

(1) 题目内容

(2) 解题过程

(3) 实验结果

2. 题目二

(1) 题目内容

(2) 解题过程

(3) 实验结果

3. 题目三

(1) 题目内容

(2) 解题过程

(3) 实验结果

4. 题目四

(1) 题目内容

(2) 解题过程

(3) 实验结果

5. 题目五

(1) 题目内容

(2) 解题过程

(3) 实验结果

6. 题目六

(1) 题目内容

(2) 解题过程

(3) 实验结果

7. 题目七

(1) 题目内容

(2) 解题过程

(3) 实验结果

8. 题目八

(1) 题目内容

(2) 解题过程

(3) 实验结果

9. 题目九

(1) 题目内容

(2) 解题过程

(3) 实验结果

10. 题目十

(1) 题目内容

(2) 解题过程

(3) 实验结果

11. 题目十一

(1) 题目内容

(2) 解题过程

(3) 实验结果

12. 题目十二

(1) 题目内容

(2) 解题过程

(3) 实验结果

13. 题目十三

(1) 题目内容

(2) 解题过程

(3) 实验结果

14. 题目十四

(1) 题目内容

(2) 解题过程

(3) 实验结果

15. 题目十五

(1) 题目内容

(2) 解题过程

(3) 实验结果

三、 解题感悟

四、 GitHub 链接和 commit 记录截图

1. GitHub 链接
2. commit 记录截图